`**修改历史**`
`**20220531  版本修改点**`
`**废弃 mAPI.installApkSilently("/sdcard/API.apk"); 函数**`
`**增加 mAPI.installApkSilently("/sdcard/API.apk","包名","类名"); 根据包名和类名自动拉起应用的接口**`

`**20220621 分类接口描述**`
`**20220625 增加 设置序列号、设置/获取设备型号、设置/获取以太网MAC地址、设置/获取IMEI号 接口**`
`**20220630 新增以太网开关和状态获取**`
`**20220701 增加 APP守护接口**`
`** 20220713 新增 设置开机图片和 开机动画 , 设置下拉状态栏状态**`

`**20220830 获取屏幕亮灭状态**`
`**20220922 修改定时开关机接口，增加几种模式**`

`**20221107 增加 APP前台守护接口**`

`**20230413 增加 单独控制屏幕显示开关**`

`**20240321 修改韦根 异步读 接口函数描述**`

`**20240521 增加 setInstallPackagePolicy接口，设置安装APK策略(白名单)**`

`**20250310 增加 设置语言接口**`

`**20251224 增加 设置看门狗超时时间**`

API接口说明

- 此文档描述的API接口，对应我司产品，需结合我司硬件规格书确定支持的硬件接口。
- 相关接口调用效果，可进入 ‘设置’ - ‘用户设置’ ，或者 ‘设置’ - ‘用户设置’ - ‘YNHCommonAPI’ 中选择对应的功能项测试 。
- 如需文档参数不全或有遗漏，具体可参考 YNHCommonAPI 源码

1： 获取接口实例

示例代码:

```
// 获取进程唯一实例
YNHAPI mAPI = YNHAPI.getInstance();
```

> 注意: 后面的所有示例代码都使用这个实例对象。

2： 系统参数

2.1： 获取主板型号

示例代码:

```
// 获取主板型号 ,返回例如ynh960 , ynh530等具体板子的型号
String model = mAPI.getBoardModel();
```

2.2：序列号

2.2.1：获取序列号

示例代码:

```
// 获取序列号。
String sn = mAPI.getSerialNo(); //返回结果与Build.SERIAL一致
```

2.2.2：设置序列号

示例代码:

```
// 设置序列号。设置后需调用重启函数，重启后方能生效
String SerialNo = "12345678";
mAPI.setSerialNo(SerialNo);
mAPI.reboot();
```

2.3：以太网MAC地址

2.3.1：获取以太网MAC地址

示例代码:

```
// 获取以太网MAC地址。
String mac = mAPI.getEthernetMAC(); //返回结果与网络服务模块返回的以太网一致
```

2.3.2：设置以太网MAC地址

示例代码:

```
// 设置以太网MAC地址。设置后需调用重启函数，重启后方能生效
String mac = "00:E0:EC:37:2A:42";
mAPI.setEthernetMAC(mac);
mAPI.reboot();
```

2.4：4G模块IMEI

2.4.1：获取4G模块IMEI

示例代码:

```
// 获取4G模块IMEI。
String imei = mAPI.getIMEI(); //返回结果与网络服务模块返回的以太网一致
```

2.4.2：设置4G模块IMEI

示例代码:

```
// 设置4G模块IMEI。设置后需调用重启函数，重启后方能生效
String imei = "864793030776509";
mAPI.setIMEI(imei);
mAPI.reboot();
```

2.5：设备型号

2.5.1：获取设备型号

示例代码:

```
// 获取设备型号。
String model = mAPI.getProductModel(); //返回结果与 Build.MODEL一致
```

2.5.2：设置设备型号

示例代码:

```
// 设置设备型号。设置后需调用重启函数，重启后方能生效
String model = "123456";
mAPI.setProductModel(model);
mAPI.reboot();
```

2.6：设备存储信息

相关数据类描述:

```
//设备存储信息(路径、空间大小等)
public class StorageInfo{
        public final static int TYPE_MEMORY = 1;
        public final static int TYPE_LOCAL_STORAGE = 2;
        public final static int TYPE_TFCARD = 3;
        public final static int TYPE_USB_STORAGE = 4;
        private int mType = 0;
        private long mTotalSize = 0; //单位 KB
        private long mFreeSize = 0; //单位 KB
        private String mPath = "";
}
```

示例代码:

```
// 获取设备的存储信息，包含 内部存储(EMMC)、TF/SD卡、USB存储（U盘，移动硬盘等）
List<StorageInfo> list =  mAPI.getStorageInfos();
for(StorageInfo info : list){
}
```

2.7：设置开机图片

示例代码:

```
mAPI.setBootLogo("/sdcard/boot.bmp"); //开机图片必须是8位的bmp位图
```

2.8：设置开机动画

示例代码:

```
mAPI.setBootAnimation("/sdcard/bootanimation.zip"); //开机动画必须标准的android动画制作，zip压缩包的名称可以随意修改
```

2.9：设置安装APK策略(白名单)

```
boolean setInstallPackagePolicy(YNHAPI.InstallPackagePolicy policy);
```

参数:

YNHAPI.InstallPackagePolicy policy： 需要设置的安装APK策略(白名单)，参数详见如下示例代码

示例代码:

```
    YNHAPI.InstallPackagePolicy policy =  new YNHAPI.InstallPackagePolicy();

    List<String> allowInstallPackageList = new ArrayList<>();
    int allowInstallType = 0;
    String allowInstallPassword = null;

    allowInstallPackageList.add("com.test.apk");
    allowInstallType = 0;
    allowInstallPassword = "1234abcd";

    /**
     * 允许安装的APK白名单，包名列表，允许allowInstallPackageList为空，为空则表示所有APK都禁止安装，或需要输入密码才能安装
     */
    policy.setAllowInstallPackageList(allowInstallPackageList);
    /**
     * 允许安装的APK白名单，使用方式。
     *  mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_NORMAL ，表示取消设置
     *  mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_NOT_ALLOW ，表示只允许安装mAllowInstallPackageList的APK，否则弹拒绝安装提示框
     *  mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_PASSWORD ，表示只允许安装mAllowInstallPackageList的APK，否则弹输入密码提示框，要求用户输入密码(mAllowInstallPassword)，密码验证通过后可继续安装
     */
    policy.setAllowInstallType(allowInstallType);
    /**
     * 允许安装APK时，需要使用的密码
     */
    policy.setAllowInstallPassword(allowInstallPassword);

    mAPI.setInstallPackagePolicy(YNHAPI.InstallPackagePolicy policy);
```

2.10：设置语言

示例代码:

```
String language="zh";
String country="CN";
mAPI.updateLanguage(language,country);
常用如下
中文（中国）：values-zh-rCN
中文（台湾）：values-zh-rTW
中文（香港）：values-zh-rHK

英语（美国）：values-en-rUS
英语（英国）：values-en-rGB
```

英文（澳大利亚）：values-en-rAU
英文（加拿大）：values-en-rCA
英文（爱尔兰）：values-en-rIE
英文（印度）：values-en-rIN
英文（新西兰）：values-en-rNZ
英文（新加坡）：values-en-rSG
英文（南非）：values-en-rZA

阿拉伯文（埃及）：values-ar-rEG
阿拉伯文（以色列）：values-ar-rIL
保加利亚文： values-bg-rBG
加泰罗尼亚文：values-ca-rES
捷克文：values-cs-rCZ
丹麦文：values-da-rDK
德文（奥地利）：values-de-rAT
德文（瑞士）：values-de-rCH
德文（德国）：values-de-rDE
德文（列支敦士登）：values-de-rLI
希腊文：values-el-rGR
西班牙文（西班牙）：values-es-rES
西班牙文（美国）：values-es-rUS
芬兰文（芬兰）：values-fi-rFI
法文（比利时）：values-fr-rBE
法文（加拿大）：values-fr-rCA
法文（瑞士）：values-fr-rCH
法文（法国）：values-fr-rFR
希伯来文：values-iw-rIL
印地文：values-hi-rIN
克罗里亚文：values-hr-rHR
匈牙利文：values-hu-rHU
印度尼西亚文：values-in-rID
意大利文（瑞士）：values-it-rCH
意大利文（意大利）：values-it-rIT
日文：values-ja-rJP
韩文：values-ko-rKR
立陶宛文：valueslt-rLT
拉脱维亚文：values-lv-rLV
挪威博克马尔文：values-nb-rNO
荷兰文(比利时)：values-nl-BE
荷兰文（荷兰）：values-nl-rNL
波兰文：values-pl-rPL
葡萄牙文（巴西）：values-pt-rBR
葡萄牙文（葡萄牙）：values-pt-rPT
罗马尼亚文：values-ro-rRO
俄文：values-ru-rRU
斯洛伐克文：values-sk-rSK
斯洛文尼亚文：values-sl-rSI
塞尔维亚文：values-sr-rRS
瑞典文：values-sv-rSE
泰文：values-th-rTH
塔加洛语：values-tl-rPH
土耳其文：values--r-rTR
乌克兰文：values-uk-rUA

```
越南文：values-vi-rVN
```

3：OTA升级

`void otaUpdate(String otaFilePath)`

参数:

- otaFilePath : ota包路径。

> 注意：升级包必须在sdcard目录下，并且名字为update.zip

示例代码:

```
mAPI.otaUpdate("/sdcard/update.zip");
```

4：APK

4.1：应用开机自启动

`void setBootLaunchApk(String packageName, boolean launch)`

参数:

- packageName : 需要开机自启动的应用包名。
- launch : true，表示开机自启动。false，表示取消开机自启动。

示例代码:

```
mAPI.setBootLaunchApk("com.innohi.helloworld", true);
```

#

4.2：静默安装APK,安装完自动启动

`void installApkSilently(String apkPath,String packageName,String className)`

参数:

- apkPath : APK的路径。
- packageName : 需要拉起的APK完整包名。例如APK包名 com.example
- className : 需要拉起的APK完整类名。例如APK类名 com.example.MainActivity

示例代码:

```
// 静默安装APK
mAPI.installApkSilently("/sdcard/API.apk","com.example","com.example.MainActivity");

如果后两个参数为null ，则表示不需要自动拉起静默安装的应用
mAPI.installApkSilently("/sdcard/API.apk",null,null);
```

4.3：卸载APK

`void uninstallApkSilently(String packageName)`

参数:

- packageName : APK的包名。

示例代码:

```
mAPI.uninstallApkSilently("com.innohi.api");
```

4.4：守护APK

A ：非前台守护

`void setAppKeepLive(String packageName, int keepAliveTimeSec)`

重启不保留，即重启后需要APK再次调用

参数:

- packageName : APK的包名。
- keepAliveTimeSec: APK 检测超时时间，单位秒，超过此时间未检测到APK活着则手动拉起APK

示例代码:

```
mAPI.setAppKeepLive("com.innohi.api", 10);
//取消非前台守护
mAPI.setAppKeepLive("", 1);
```

B ：前台守护

`void setForegroundAppKeepLive(String packageName, int keepAliveTimeSec)`

重启不保留，即重启后需要APK再次调用

参数:

- packageName : APK的包名。

- keepAliveTimeSec：APK 检测超时时间，单位秒，超过此时间未检测到APK活着则手动拉起APK

示例代码:

```
mAPI.setForegroundAppKeepLive("com.innohi.api", 10);
//取消前台守护
mAPI.setForegroundAppKeepLive("", 1);
```

5：显示相关

5.1：获取屏幕旋转角度

`RotationDegree getScreenRotation(ScreenType screenType)`

参数:

- screenType : 表示获取哪一个屏幕的旋转角度。例如，ScreenType.MAIN 表示主屏幕，ScreenType.AUX 表示副屏幕。

返回值:

- RotationDegree : 返回指定屏幕的旋转角度。例如，RotationDegree.DEGREE_90 表示90度。

示例代码:

```
YNHAPI.RotationDegree screenRotation
            = mAPI.getScreenRotation(YNHAPI.ScreenType.MAIN);
if (screenRotation == YNHAPI.RotationDegree.DEGREE_90) {
    Log.d(TAG, "主屏幕90度");
}
```

5.2：设置屏幕旋转角度

`void setScreenRotation(ScreenType type, RotationDegree degree)`

参数:

- type : 设置设置哪个屏幕的旋转角度。例如，ScreenType.MAIN 表示主屏幕。
- degree : 需要设置的角度。例如，RotationDegree.DEGREE_90 表示90度。

示例代码:

```
// 设置副屏旋转180度
mAPI.setScreenRotation(YNHAPI.ScreenType.AUX,
                           YNHAPI.RotationDegree.DEGREE_180);
```

5.3：获取屏幕密度

`LcdDensity getLcdDensity()`

返回值:

- LcdDensity : 屏幕密度。例如，LcdDensity.DENSITY_160 表示屏幕密度为 160 dpi。

示例代码:

```
YNHAPI.LcdDensity density = mAPI.getLcdDensity();
if (density == YNHAPI.LcdDensity.DENSITY_160) {
    Log.d(TAG, "屏幕密度为160dpi");
}
```

5.4：设置屏幕密度

`void setLcdDensity(LcdDensity density)`

参数:

- density : 表示需要设置的屏幕密度。例如，LcdDensity.DENSITY_160 表示设置屏幕密度为 160 dpi。

示例代码:

```
// 设置屏幕密度为240dpi
mAPI.setLcdDensity(YNHAPI.LcdDensity.DENSITY_240);
```

5.5：获取屏幕亮灭状态

`boolean isScreenOn()`

参数:

- 无
- **不能用于休眠/唤醒 sleep/wakeup 之后判断屏幕亮灭**
- **可用于 APP调用 setScreenOnOff 自主设置屏幕开关**

示例代码:

```
// 屏幕亮为true，灭为false
mAPI.isScreenOn();
```

5.6：设置屏幕开关

`void setScreenOnOff(boolean on)`

参数:

- boolean on 屏幕是否显示
- **用于非标准的休眠/唤醒 sleep/wakeup 的场景，APP可单独控制屏幕开关**
- **通过 isScreenOn 判断屏幕开关状态**

示例代码:

```
// 屏幕亮为true，灭为false
mAPI.setScreenOnOff(false);
```

6：看门狗

6.1：打开/关闭看门狗

`void enableWatchdog(boolean enable)`

参数:

- enable : true，表示打开看门狗，false，表示关闭看门狗。

示例代码:

```
// 打开看门狗
mAPI.enabledWatchdog(true);
// 关闭看门狗
mAPI.enableWatchdog(false);
```

6.2：喂狗

`void feedWatchdog()`

示例代码:

```
// 喂狗
// 打开看门狗后，最多15秒内如果没喂狗就会自动复位，建议10秒左右调用一次feedWatchdog 函数喂看门狗
mAPI.feedWatchdog();
```

6.3：设置看门狗超时时间

`void setWatchdogTimeout(int second)`

示例代码:

```
// 设置 15秒后超时，不同平台可能时间有1-2秒差异
mAPI.setWatchdogTimeout(15);
```

#

7：关机与重启

示例代码:

```
// 关机
mAPI.shutdown();
// 重启
mAPI.reboot();
```

8：系统休眠与唤醒

示例代码:

```
//通过PowerManager 判断系统休眠唤醒状态
PowerManager pm = (PowerManager) mSystemContext.getSystemService(Context.POWER_SERVICE);
boolean isInteractive = pm.isInteractive(); //Android 11 及后续Android版本接口
// 休眠
mAPI.sleep();
// 唤醒
mAPI.wakeup();
```

9：导航栏和下拉状态栏

9.1：设置导航栏可见性

`void setNavigationBarVisibility(NavigationBarVisibility visibility)`

参数:

- visibility : 导航栏可见性。有三种值，如下
    - NavigationBarVisibility.INVISIBLE :导航栏不可见，但是可以从屏幕底部滑出导航栏。
    - NavigationBarVisibility.VISIBLE : 导航栏可见。
    - NavigationBarVisibility.ALWAYS_INVISIBLE : 导航栏不可见，也不可以从屏幕底部滑出导航栏。

示例代码:

```
mAPI.setNavigationBarVisibility(YNHAPI.NavigationBarVisibility.ALWAYS_INVISIBLE);
```

9.2：获取导航栏可见性

`NavigationBarVisibility getNavigationBarVisibility()`

返回值:

- NavigationBarVisibility : 导航栏可见性。有三种值，如下
    - NavigationBarVisibility.INVISIBLE :导航栏不可见，但是可以从屏幕底部滑出导航栏。
    - NavigationBarVisibility.VISIBLE : 导航栏可见。
    - NavigationBarVisibility.ALWAYS_INVISIBLE : 导航栏不可见，也不可以从屏幕底部滑出导航栏。

示例代码:

```
// 获取导航栏可见性。
YNHAPI.NavigationBarVisibility visibility = mAPI.getNavigationBarVisibility();
// 如果导航栏始终不可见，那么就显示导航栏。
if (visiblity == YNHAPI.NavigationBarVisibility.ALWAYS_INVISIBLE) {
    mAPI.setNavigationBarVisibility(YNHAPI.NavigationBarVisibility.VISIBLE);
}
```

9.3：设置下拉状态栏可见性

`void setExtendStatusBarVisibility(ExtendStatusBarVisibility visibility)`

示例代码:

```
// 设置完成后，需要重启设备，下次开机后设置才能生效
mAPI.setExtendStatusBarVisibility(YNHAPI.ExtendStatusBarVisibility.VISIBLE); //可下拉顶部状态栏
```

9.4：获取下拉状态栏可见性

`ExtendStatusBarVisibility  getExtendStatusBarVisibility()`

返回值:

- ExtendStatusBarVisibility : 状态栏可见性。有两种值，如下
- ExtendStatusBarVisibility.INVISIBLE :状态栏不可下拉
- ExtendStatusBarVisibility.VISIBLE : 状态栏可下拉

示例代码:

```
// 获取下拉状态栏可见性。
YNHAPI.ExtendStatusBarVisibility  visibility = mAPI.getExtendStatusBarVisibility();
```

10：root权限

10.1：检测root

`boolean isRoot()`

返回值:

- boolean : true，表示可以通过 su 获取 root 权限。否则不可以。

10.2：打开/关闭root

`boolean enableRoot(boolean enable)`

参数:

- enable : true，表示打开 root。false，表示关闭 root。

返回值:

- boolean : true，表示成功。false，表示失败。

示例代码:

```
// 如果root权限关闭了，就打开root权限
if (!mAPI.isRoot()) {
    if (mAPI.enableRoot(true)) {
        Log.d(TAG, "成功root");
    }
}
```

11：网络相关

11.1：以太网

11.1.1：获取IP信息

`IpConfig getIpConfig()`

返回值:

- IpConfig : 代表IP信息的一个实体类。

示例代码:

```
// 获取IP信息
IpConfig ipConfig = mAPI.getIpConfig();
// IP地址
String ip = ipConfig.ip;
// 子网掩码
String mask = ipConfig.mask;
// 网关
String gateway = ipConfig.gateway;
// dns列表
ArrayList<String> dnsList = ipConfig.dnsList;
```

11.1.2：使用以太网静态IP

`void setStaticIp(IpConfig config)`

参数:

- config : 表示需要设置静态IP的信息。

示例代码:

```
mAPI.setStaticIp(new IpConfig("192.168.1.2", "255.255.255.0", "192.168.8.1",
Arrays.asList("192.168.8.1")));
```

11.1.3：使用以太网动态IP

`void setDhcpIp()`

示例代码:

```
// 设置使用动态IP
mAPI.setDhcpIp();
```

11.1.4：获取以太网IP模式

`IpMode getIpMode()`

返回值:

- IpMode : 表示IP模式。例如, IpMode.STATIC 表示静态IP模式，IpMode.DHCP 表示动态IP模式。

示例代码:

```
IpMode mode = mAPI.getIpMode();
if (mode == IpMode.STATIC) {
    Log.d(TAG, "静态IP模式");
} else if (mode == IpMode.DHCP) {
    Log.d(TAG, "动态IP模式");
}
```

11.1.5：以太网是否打开

`boolean isEthernetOpen()`

返回值:

- boolean ：true为已打开，false 为关闭

示例代码:

```
boolean isOpen = mAPI.isEthernetOpen();
```

11.1.6：开关以太网

`void setEthernetState(boolean open)`

参数:

- open ： true为打开以太网，false为关闭以太网

示例代码:

```
mAPI.setEthernetState(true);
```

12：系统时间

12.1：设置系统时间

`void setSystemTime(long timeInMills)`

参数:

- timeInMills : 表示需要设置的系统时间，单位为毫秒。

示例代码:

```
// 设置系统时为2021-9-28 6:0:0
// 注意Calendar类月份从0开始，日从1开始
Calendar calendar = Calendar.getInstance(Locale.getDefault());
calendar.set(2021, 10, 28, 6, 0, 0);
mAPI.setSystemTime(calendar.getTimeInMillis());
```

12.2：设置定时开关机

void setPowerOnOffAlarmCycle(int type ,int[] timeOff, int[] timeOn)`

参数:

- type: 需要设置开关机类型。
- timeOff : 需要设置的定时关机时间。
- timeOn : 需要设置的定时开机时间。

示例代码:

```
// type = 1 设定开关机时间，只有一次生效 ,需每次开机客户APP自己再次设置
// type = 3 以周为周期设置开关机时间，设置完成后系统保存并生效，恢复出厂设置或刷固件需要重新设置

int type = 1;
int[] weekdays= null;//此值为null或者随便设置都可以，在type 为非3时，weekdays无效
int[] poweroff={2017,3,16,20,30};//即在2017/3/16  20:30关机,小时取值0-23,分钟取值0-59
int[] poweron={2017,3,17,8,35};//即在2017/3/17  8:35开机,小时取值0-23,分钟取值
mAPI.setPowerOnOffAlarmCycle(type,weekdays,poweroff,poweron);

int type = 3;
int[] weekdays={1,3,4,5};//设置周一、周三、周四、周五定时开关机,必选项
int[] poweroff={20,30};//设置关机时间为20:30,小时取值0-23,分钟取值0-59
int[] poweron={8,35};//设置开机时间为8:35,小时取值0-23,分钟取值0-59
mAPI.setPowerOnOffAlarmCycle(type,weekdays,poweroff,poweron);
```

12.3：取消定时开关机

`void cancelPowerOnOffAlarm()`

示例代码:

```
mAPI.cancelPowerOnOffAlarm();
```

12.4：网络同步时间

12.4.1：是否使用网络同步时间

`void isEnableNetworkProvidedTime()`

示例代码:

```
boolean isEnable = mAPI.isEnableNetworkProvidedTime();
```

12.4.2：设置网络同步时间

`void setEnableNetworkProvidedTime(boolean enable)`

示例代码:

```
// 设置是否允许网络同步时间，true为允许，false 为不允许
mAPI.setEnableNetworkProvidedTime(true);
```

13：硬件模块

13.1：GPIO

所有通过GPIO控制的设备，本节的API都适用。例如，IO1，继电器，LED灯，USB供电（GPIO口控制USB口供电）等

注意板子支持的外设，如继电器，LED等，GPIO口等，需要查询 具体的板子规格书 ，使用时传的参数可注意我司提供的YNHAPI.jar 里面的内部成员变量，具体详细实现可参考我司提供的YNHCommonApiAPK源码

可操作的外设IO口如下，（有些板子不支持，调用可能没效果），详细见 YNHAPI.jar

```
    const int GPIO_1            =   0;
    const int GPIO_2            =   1;
    const int GPIO_3            =   2;
    const int GPIO_4            =   3;
    const int GPIO_5            =   4;
    const int GPIO_6            =   5;
    const int GPIO_7            =   6;
    const int GPIO_8            =   7;
    const int GPIO_9            =   8;
    const int GPIO_10           =   9;
    const int GPIO_11           =   10;
    const int GPIO_12           =   11;
    const int GPIO_13           =   12;
    const int GPIO_14           =   13;
    const int GPIO_15           =   14;
    const int LIGHT_RED         =   15; 红色补光灯
    const int LIGHT_GREEN       =   16; 绿色补光灯
    const int LIGHT_BLUE        =   17; 蓝色补光灯
```

```
    const int LIGHT_WHITE       =   18; 白色补光灯
    const int LIGHT_INFRARED    =   19; USB摄像头红外补光灯
    const int RELAY             =   20; 继电器
    const int BELL              =   21; 门铃
    const int CASHBOX_0         =   22; 第一个钱箱
    const int CASHBOX_1         =   23; 第二个钱箱
    const int LIGHT_BOARD_BLUE  =   24; 板载蓝灯
    const int USB_1             =   25; 第一个USB供电
    const int USB_2             =   26; 第二个USB供电
    const int USB_3             =   27; 第三个USB供电
    const int LIGHT_BOARD_RED  =   28; 板载红灯
    const int LIGHT_CAMERA_EXTERNAL_INFRARED  =   29; MIPI摄像头红外补光灯
    const int LIGHT_BOARD_YELLOW  =   30;
```

13.1.1：获取GPIO状态

`GpioState getGpioState(int index)`

参数:

- index : 表示需要获取状态的GPIO。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红灯等，详细可参考我司提供的YNHAPI.jar。

示例代码:

```
if (mAPI.getGpioState(YNHAPI.GPIO_1) == YNHAPI.GpioState.HIGH) {
    Log.d(TAG, "GPIO1为高电平");
}
```

13.1.2：设置GPIO状态

`boolean setGpioState(int index, GpioState state)`

参数:

- index : 表示需要设置状态的GPIO。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红灯等，详细可参考我司提供的YNHAPI.jar。
- state : 需要设置的状态。例如，GpioState.HIGH，表示设置为高电平。

返回值:

- boolean : true，表示成功，false，表示失败。

示例代码:

```
if (mAPI.setGpioState(YNHAPI.GPIO_1, YNHAPI.GpioState.HIGH)) {
    Log.d(TAG, "成功设置高电平");
}
```

13.1.3：获取GPIO模式

`GpioMode getGpioMode(int index)`

参数:

- index : 表示需要获取模式的GPIO。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红灯，详细可参考我司提供的YNHAPI.jar。

返回值:

- GpioMode : 表示获取的GPIO模式。例如，GpioMode.INPUT，表示输入模式。

示例代码:

```
YNHAPI.GpioMode gpioMode = mAPI.getGpioMode(YNHAPI.GPIO_1);
if (gpioMode == GpioMode.INPUT) {
    Log.d(TAG, "GPIO1为输入模式");
}
```

13.1.4：设置GPIO模式

`boolean setGpioMode(int index, GpioMode mode)`

参数:

- index : 表示需要设置模式的GPIO索引。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红灯，详细可参考我司提供的YNHAPI.jar。
- mode : 表示需要设置的模式。例如，GpioMode.INPUT，设置为输入模式。

返回值:

- boolean : true，表示成功，false，表示失败。

示例代码:

```
if (mAPI.setGpioMode(YNHAPI.GPIO_1, YNHAPI.GpioMode.INPUT)) {
    Log.d(TAG, "成功设置输入模式");
}
```

13.1.4：监听GPIO状态改变

`synchronized void listenGpio(int index, GpioListenerCallback callback)`

此函数调用后内部会每隔1秒检测下GPIO状态，如果有变化则会通知应用，故此这个函数比较适合GPIO变化后能持续保持一段时间的场景。并不适合检测按键短按，或者变化时间在1秒以内的场景，如有这种场景可以直接使用getGpioState，并由客户自己控制周期

参数:

- index : 表示需要监听状态改变的GPIO索引。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红

灯，详细可参考我司提供的YNHAPI.jar。
- callback : 监听的回调。

示例代码:

```
YNHAPI.GpioListenerCallback gpioListenerCallback = new YNHAPI.GpioListenerCallback() {
    public void onChanged(int index, int oldValue, int newValue) {
        // 当状态发生改变时，回调此函数
        // 参数index是GPIO索引值，表示哪个GPIO电平状态发生改变
        // 参数oldValue表示旧的电平状态
        // 参数newValue表示新的电平状态
    }
};
mAPI.listenGpio(YNHAPI.GPIO_1, gpioListenerCallback);
```

13.1.5：取消监听GPIO状态改变

`synchronized void unlistenGpio(int index, GpioListenerCallback callback)`

参数:

- index : 表示需要取消监听状态改变的GPIO索引。例如GPIO_1表示IO1，RELAY表示继电器，LIGHT_RED 表示红灯，详细可参考我司提供的YNHAPI.jar。
- callback : 已经设置的监听的回调。

示例代码:

```
// 监听GPIO1状态改变
mAPI.listenGpio(YNHAPI.GPIO_1, gpioListenerCallback);

// 当不再需要监听GPIO1电平状态改变时，调用unlistenGpio()
mAPI.unlistenGpio(YNHAPI.GPIO_1, gpioListenerCallback);
```

13.2：设置LED灯亮度

`void setLightBrightness(int index, int brightness)`

参数:

- index : LED对应的GPIO索引。 例如，LIGHT_RED, LIGHT_GREEN, LIGHT_WHITE，详细可参考我司提供的YNHAPI.jar。
- brightness : 亮度值，范围为[0, 255]。

示例代码:

```
// 设置红灯的亮度为80%
mAPI.setLightBrightness(YNHAPI.LIGHT_RED, 204);
```

13.3：韦根

13.3.1：获取韦根模式

`WiegandMode readWiegandMode()`

返回值:

- WiegnadMode : 获取的韦根模式。例如，WiegandMode.INPUT 表示输入模式。

示例代码:

```
YNHAPI.WiegandMode wiegandMode = mAPI.readWiegandMode();
if (wiegandMode == YNHAPI.WiegandMode.INPUT) {
    Log.d(TAG, "韦根模式为输入模式");
}
```

13.3.2：设置韦根模式

`void writeWiegandMode(WiegandMode mode)`

参数:

- mode : 需要设置的韦根模式。例如，WiegandMode.INPUT，表示设置为输入模式。

示例代码:

```
mAPI.writeWiegandMode(YNHAPI.WiegandMode.INPUT);
```

13.3.3：读韦根

韦根的分为同步阻塞读，和异步回调读。

13.3.4：同步阻塞读

`long readWiegand()`

返回值:

- long : 读取的韦根号。

示例代码:

```
CompletableFuture.supplyAsync(() -> mAPI.readWiegand()) // 异步线程读取
        .whenComplete((code, throwable) -> {
            // 主线程更新
            runOnUiThread(() -> mTextView.setText(String.valueOf(code)));
        });
```

13.3.5：异步读

`void readWiegandAsyn(WiegandCallback callback)`

参数:

- callback : 异步读取韦根的回调。

示例代码:

注意需要按照如下调用方式调用。如果需要多次异步读取韦根卡号，一定要在 onSuccess或者onFailure 回调函数里面调用 readWiegandAsync。

```
private YNHAPI.WiegandCallback mCallback = new YNHAPI.WiegandCallback() {
        @Override
        public void onSuccess(long code) {
            // 成功读取韦根号，参数 code 表示韦根号。

            //如果需要循环读取韦根，需要重复调用 readWiegandAsync
            mAPI.readWiegandAsync(mCallback);
        }

        @Override
        public void onFailure() {
            // 读取失败

            //如果需要循环读取韦根，需要重复调用 readWiegandAsync
            mAPI.readWiegandAsync(mCallback);
        }
};
mAPI.readWiegandAsyn(mCallback);
```

> 注意，回调发生在工作线程。

13.3.6：写韦根

`void writeWiegand(WiegandFormat format, long code)`

参数:

- format : 韦根格式。例如，WiegandFormat.FORMAT_26，表示26位韦根格式。
- code : 韦根码。

示例代码:

```
mAPI.writeWiegand(YNHAPI.WiegandFormat.FORMAT_26, 12345678);
```

14：功能参考代码

14.1：钱箱功能

钱箱功能就是操作GPIO，先把GPIO状态拉低，过500ms后，再拉高。

示例代码:

```
CompletableFuture.runAsync(() -> {
    mYNHAPI.setGpioState(YNHAPI.CASHBOX_0, YNHAPI.GpioState.LOW);
    try {
        TimeUnit.MILLISECONDS.sleep(500);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}).thenAccept(u -> {
    mYNHAPI.setGpioState(YNHAPI.CASHBOX_0, YNHAPI.GpioState.HIGH);
});
```

14.2：执行 root 权限命令

示例代码:

```
boolean executeCmd(String content) throws IOException, InterruptedException {
    boolean ret = false;
    Process p = Runtime.getRuntime().exec("su");
    if (p != null) {
        PrintWriter pw = new PrintWriter(p.getOutputStream(), true);
        pw.println(content);
        pw.println("exit");
        ret = p.waitFor() == 0;
    }
    return ret;
}
```